TIP101 | Intro to Technical Interview Prep

Intro to Technical Interview Prep Summer 2026 (@ Section 1 | Tuesdays and Thursdays 3PM - 5PM PT)
Personal Member ID#: **151287**

Need help? Post on our **class slack channel** or email us at **support@codepath.org**

Unit 5: Session 1

Object-Oriented Programming (OOP)

In this session, students will learn to apply Python classes and linked lists through practical exercises. They will begin by creating and manipulating instances of a Pokémon class and then explore the basics of linked lists, focusing on node creation and linkage. These exercises aim to deepen understanding of object-oriented programming and provide foundational skills in managing custom data structures in Python.

You can find all resources from today including the session deck, session recording, and more on the resources tab

📈 Part 1 : Instructor Led Session

We'll spend the first portion of the synchronous class time in large groups, where the instructor will lead class instruction for 30-45 minutes.

👩‍💻 Part 2: Breakout Session

In breakout sessions, we will explore and collaboratively solve problem sets in small groups. Here, the **collaboration, conversation, and approach** are just as important as "solving the problem" - please engage warmly, clearly, and plentifully in the process!

In breakout rooms you will:

- Screen-share the problem/s, and verbally review them together
- Screen-share an interactive coding environment, and talk through the steps of a solution approach
    - ProTip: - An Integrated Development Environment (IDE) is a fancy name for a tool you could use for shared writing of code - like Replit.com, Collabed.it, CodePen.io, or other - your staff team will specify which tool to use for this class!
- Screen-share an implementation of your proposed solution
- Independently follow-along, or create an implementation, in your own IDE.

Your program leader/s will indicate which code sharing tool/s to use as a group, and will help break down or provide specific scaffolding with the main concepts above.

▶ **Note on Expectations**

🔎 Problem Solving Approach

We will approach problems using the six steps in the UMPIRE approach.

UMPIRE: Understand, Match, Plan, Implement, Review, Evaluate.

We'll apply these six steps to the problems we'll see in the first half of the course.

We will learn to:

- **Understand** the problem
- **Match** identifies common approaches you've seen/used before
- **Plan** a solution step-by-step, and
- **Implement** the solution
- **Review** your solution

- **Evaluate** your solution's time and space complexity and think critically about the advantages and disadvantages of your chosen approach.

✴ **Unit 5 Cheatsheet**

To jumpstart your journey into object-oriented programming and linked lists, we've put together a guide to common concepts and syntax you will use throughout Unit 5 breakout problems. Use this cheatsheet as a quick reference guide as you work through the problems below.

Problem Set Version 1

▼ Problem 1: Pokemon Class

Step 1: Copy the following code into your IDE.

Step 2: Add a line of code (outside of the class) to instantiate an instance of the class `Pokemon` and store it in a variable named `my_pokemon`. The Pokemon instance created should have `name` `"Pikachu"` and its `types` should be `["Electric"]`.

```
class Pokemon:
    def __init__(self, name, types):
        self.name = name
        self.types = types
        self.is_caught = False
```

▼ ✨ AI Hint: Intro to Object Oriented Programming

Key Skill: Use AI to explain code concepts

This problem may require you to be familiar with Object Oriented Programming (OOP) basics, including classes, instances, objects, and constructors. To help, we've included an "intro to OOP" review Unit 5 Cheatsheet

You can also use an AI tool like ChatGPT or GitHub Copilot to get more examples or ask follow-up questions. You can use the following prompt as a starting point:

"You're an expert computer science tutor. Can you help me understand OOP conceptually, using analogies to real-world objects?"

Once you understand the concept, you can also ask follow-up questions like:

"Can you provide an example of a class, instance, and constructor in python?"

"What does `self` mean in Python, and how is it used in OOP?"

Close Section

▼ Problem 2: Create Squirtle

Step 1: Add the `print_pokemon` definition below to your code on your IDE.

Step 2: Instantiate an instance of the class `Pokemon` and store it in a variable named `squirtle`. The Pokemon instance created should have `name` "Squirtle" and its `types` should be ["Water"].

Step 3: Call the method `print_pokemon()` on your new Pokemon instance `squirtle`.

```
class Pokemon:
    def __init__(self, name, types):
        self.name = name
        self.types = types
        self.is_caught = False

    def print_pokemon(self):
        print({
            "name": self.name,
            "types": self.types,
            "is_caught": self.is_caught
        })
```

Example Usage:

```
squirtle = Pokemon("Squirtle", ["water"])
squirtle.print_pokemon()
```

Example Output:

```
{'name': 'Squirtle', 'types': ['water'], 'is_caught': False}
```

▼ ✨ AI Hint: Class Methods

Key Skill: Use AI to explain code concepts

This question requires you to be familiar with class methods, which are functions attached to an object. To help, we've included more info Unit 5 Cheatsheet

If you'd still like to see more examples or ask follow-up questions, try using an AI tool like ChatGPT or GitHub Copilot. You can use the following prompt as a starting point:

"You're an expert computer science tutor. Please provide 2-3 examples of how Class Methods are used in Python, and explain how each one works."

You might also want to ask questions like:

"Can you explain the difference between class methods, instance methods, and functions?"

Close Section

▼ Problem 3: Is Caught

Using your code from Problem 2, update your `squirtle` Pokemon so that `is_caught` is updated to `True`. Use the `print_pokemon()` function to verify that `squirtle`'s `is_caught` property was updated.

Expected Output:

```
{
    "name": "Squirtle",
```

```
    "types": ["Water"],
    "is_caught": True
}
```

Example Output

```
{'name': 'Squirtle', 'types': ['water'], 'is_caught': True}
```

▼ ✨ AI Hint: Class Attributes

Key Skill: Use AI to explain code concepts

This problem may require you to be familiar with class attributes, which are variables attached to an object. To help, we've included more info Unit 5 Cheatsheet

If you'd still like to see more examples or ask follow-up questions, try using an AI tool like ChatGPT or GitHub Copilot. You can use the following prompt as a starting point:

"You're an expert computer science tutor. Please provide 2-3 examples of how Class Attributes are used in Python, and explain how each one works."

Close Section

▼ Problem 4: Catch Pokemon

Update the `Pokemon` class with a new method `catch()` that takes in no parameters except `self`.

The method should update the Pokemon's `is_caught` attribute to `True` and not return any value.

```
class Pokemon():
        ...

        def catch(self):
                pass
```

Example Usage:

```
my_pokemon = Pokemon("rattata", ["Normal"])
my_pokemon.print_pokemon()

my_pokemon.catch()
my_pokemon.print_pokemon()
```

Example Output:

```
{'name': 'rattata', 'types': ['Normal'], 'is_caught': False} # First print statement
{'name': 'rattata', 'types': ['Normal'], 'is_caught': True}  # Second print statement
```

▼ ✨ **AI Hint: Writing Methods**

Key Skill: Use AI to explain code concepts

This problem requires you to write your own method! Try it yourself, but if you get stuck, you can:

- Check out the Unit 5 Cheatsheet
- Use an AI tool like ChatGPT or GitHub Copilot to show you examples of how to write methods in Python

Close Section

▼ Problem 5: Choose Pokemon

Update the `Pokemon` class with a new method `choose()` that takes in no parameters except `self`.

If the Pokemon is caught, the method should print the string `"<Pokemon name> I choose you!"`.

Otherwise, it should print `"<Pokemon name> is wild! Catch them if you can!"`.

```
class Pokemon():
        ...

        def choose(self):
                pass
```

Example Usage:

```
my_pokemon = Pokemon("rattata", ["Normal"])
my_pokemon.print_pokemon()

my_pokemon.choose()
my_pokemon.catch()
my_pokemon.choose()
```

Example Output:

```
{'name': 'rattata', 'types': ['Normal'], 'is_caught': False}
rattata is wild! Catch them if you can!
rattata I choose you!
```

Close Section

▼ Problem 6: Add Pokemon Type

Update the `Pokemon` class with a new method `add_type()` that takes in a string `new_type` as a parameter.

It should add `new_type` to the Pokemon's list of `types`.

```
class Pokemon():
        ...

        def add_type(self, new_type):
                pass
```

Example Usage:

```
jigglypuff = Pokemon("Jigglypuff", ["Normal"])
jigglypuff.print_pokemon()

jigglypuff.add_type("Fairy")
jigglypuff.print_pokemon()
```

Example Output:

```
{'name': 'Jigglypuff', 'types': ['Normal'], 'is_caught': False}
{'name': 'Jigglypuff', 'types': ['Normal', 'Fairy'], 'is_caught': False}
```

Close Section

▼ Problem 7: Get Pokemon

Outside the Pokemon class, write a new function `get_by_type()` that takes in a list of Pokemon instances `my_pokemon` and a string `pokemon_type` as parameters.

The function should return a list of all Pokemon instances from `my_pokemon` that have the type `pokemon_type`.

Hint: To test, loop over Pokemon in return list and print the Pokemon's name

```
class Pokemon():
        ...

def get_by_type(my_pokemon, pokemon_type):
        pass
```

Example Usage:

```
# initializing pokemons
jigglypuff = Pokemon("Jigglypuff", ["Normal", "Fairy"])
diglett = Pokemon("Diglett", ["Ground"])
meowth = Pokemon("Meowth", ["Normal"])
pidgeot = Pokemon("Pidgeot", ["Normal", "Flying"])
blastoise = Pokemon("Blastoise", ["Water"])

my_pokemon = [jigglypuff, diglett, meowth, pidgeot, blastoise]
```

```
normal_pokemon = get_by_type(my_pokemon, "Normal")
print(normal_pokemon)
```

Example Output: `[Jigglypuff, Meowth, Pidgeot]`

Close Section

▼ Problem 8: Pokemon Evolution

Some Pokemon can evolve into other species of Pokemon. In the updated `Pokemon` class below, each instance of `Pokemon` has an attribute `evolution`. The attribute will either be the default value of `None` or another `Pokemon` instance.

Write a function `get_evolutionary_line()` that takes in a `Pokemon` object `starter_pokemon` as a parameter.

The function should return a list of itself and the Pokemon that the `starter_pokemon` can evolve into.

```
class Pokemon():
        def  __init__(self, name, types, evolution = None):
                self.name = name
                self.types = types
                self.is_caught = False
                self.evolution = evolution

def get_evolutionary_line(starter_pokemon):
        pass
```

Example Usage:

```
charizard = Pokemon("Charizard", ["fire", "flying"])
charmeleon = Pokemon("Charmeleon", ["fire"], charizard)
charmander = Pokemon("Charmander", ["fire"], charmeleon)

charmander_list = get_evolutionary_line(charmander)
print(charmander_list)

charmeleon_list = get_evolutionary_line(charmeleon)
print(charmeleon_list)
```

```
charizard_list = get_evolutionary_line(charizard)
print(charizard_list)
```

Example Output:

```
[`Charmander`, `Charmeleon`, `Charizard`]
[`Charmeleon`, `Charizard`]
['Charizard']
```

Close Section

▼ Problem 9: Node Class

A **linked list** is a new data type that, similar to a normal list or array, allows us to store pieces of data sequentially. The difference between a linked list and a normal list lies in how each element is stored in a computer's memory.

In a normal list, individual elements of the list are stored in adjacent memory locations according to the order they appear in the list. If we know where the first element of the list is stored, it's really easy to find any other element in the list.

In a linked list, the individual elements called **nodes** are not stored in sequential memory locations. Each node may be stored in an unrelated memory location. To connect nodes together into a sequential list, each node stores a reference or pointer to the next node in the list.

Using the provided `Node` class below, create two nodes:

- The first node should have value `a` and be stored in a variable `node_one`.
- The second node should have value `b` and be stored in a variable `node_two`.

You do not need to connect the nodes together yet.

```
class Node:
        def __init__(self, value, next=None):
                self.value = value
                self.next = next
```

Example Usage:

```
print(node_one.value)
print(node_one.next)
print(node_two.value)
print(node_two.next)
```

Example Output:

```
a
None
b
None
```

▼ ✨ AI Hint: Linked Lists

Key Skill: Use AI to explain code concepts

This question requires you to be familiar with Linked Lists, a incredibly useful but sometimes tricky data structure. To help, we've included a review of linked lists Unit 5 Cheatsheet

You can also use an AI tool like ChatGPT or GitHub Copilot to get more examples or ask follow-up questions. You can use the following prompt as a starting point:

"You're an expert computer science tutor. Can you help me understand linked lists conceptually, using analogies to real-world objects?"

Once you understand the concept of Linked Lists, you can also ask follow-up questions like:

"Can you provide examples of how to implement a linked list in Python, and explain how each part works?"

"Here is a provided Linked List class: (CODE). Can you give me an example of how to access the data in this linked list?"

Close Section

▼ Problem 10: Linking Nodes

Building off the `Node` class from Problem 9, now we will link the nodes together.

To link the nodes, we can set a node's `next` attribute to hold another node. Update `node_one` from the Problem 9 to point to `node_two`.

Example Usage (*after updating* `node_one`*'s* `next` *property*):

```
print(node_one.value)
print(node_one.next.value)
print(node_two.value)
```

Example Output:

```
a
b
b
```

Close Section

▼ Problem 11: Mario Party

Create the list `["Mario", "Luigi", "Wario", "Toad"]` as a linked list given the `Node` class:

```
class Node:
        def __init__(self, value, next=None):
                self.value = value
                self.next = next
```

Result Linked List would be: `Mario -> Luigi -> Wario -> Toad`

Example Usage (*after completing the problem with variable names* `node_1`, `node_2`, `node_3`, *and* `node_4`.):

```
print(node_1.value, "->", node_1.next.value)
print(node_2.value, "->", node_2.next.value)
```

```
print(node_3.value, "->", node_3.next.value)
print(node_4.value, "->", node_4.next)
```

Example Output:

```
Mario -> Luigi
Luigi -> Wario
Wario -> Toad
Toad -> None
```

Close Section

▼ Problem 12: Printing Linked List

Write a function `print_linked_list()` that takes in a head node as a parameter and prints the linked list using the string `" -> "` to separate each node.

Note: The "head" of a linked list is the first element in the linked list, equivalent to `lst[0]` of a normal list.

```
class Node:
        def __init__(self, value, next=None):
                self.value = value
                self.next= next

def print_linked_list(head):
        pass
```

Example Input:

```
# input linked list: a->b->c->d->e
print_linked_list(a)
```

Example Output:

```
a -> b -> c -> d -> e
```

▼ 💡 Hint: Linked List Traversal

This problem requires you to traverse a linked list. In other words, it requires you to iterate over the nodes of a linked list. For a break down of how to traverse a linked list, check out the Unit 5 Cheatsheet.

Close Section

Problem Set Version 2

- ▶ Problem 1: Card Class
- ▶ Problem 2: Print Card
- ▶ Problem 3: Verify Update
- ▶ Problem 4: Valid Card
- ▶ Problem 5: Get Value
- ▶ Problem 6: Hand Class
- ▶ Problem 7: Sum of Cards
- ▶ Problem 8: Print Hand
- ▶ Problem 9: Head and Tail Nodes
- ▶ Problem 10: Middle Node
- ▶ Problem 11: Zodiac Signs
- ▶ Problem 12: Print Linked List

Problem Set Version 3

- ▶ Problem 1: Player Class
- ▶ Problem 2: Get Player
- ▶ Problem 3: Update Kart
- ▶ Problem 4: Set Character

- Problem 5: Add Special Item
- Problem 6: Print Inventory
- Problem 7: Race Results
- Problem 8: Get Rank
- Problem 9: Tom and Jerry
- Problem 10: Chase List
- Problem 11: Update Chase
- Problem 12: Chase String